

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: ITA 0606

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO4: K- Nearest Neighbour Learning – Locally weighted Regression – Radial Bases Functions – Case Based Learning **(BL 5)**

Assessment Tool Weightage: 10%

QUESTIONS: 05

Total Marks:100

Annexure A

Comparative Analysis

Code of Conduct:

I Gayathri N P (1924221362) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

1. Compare K-Nearest Neighbour and Case-Based Learning using a real-world example such as medical diagnosis. Analyze differences in memory usage, similarity measures, and prediction strategy.

Answer :

Introduction

K-Nearest Neighbour (KNN) and Case-Based Learning (CBL) are instance-based learning techniques. Both methods solve a new problem by using previously stored examples or cases. They are useful in medical diagnosis because a new patient's symptoms can be compared with previous patients.

K-Nearest Neighbour

KNN classifies a new patient by finding the **K most similar patients** in the training dataset. Similarity is usually calculated using distance measures such as **Euclidean distance**.

For example, if a patient has fever, cough, temperature and blood pressure values similar to patients previously diagnosed with pneumonia, KNN can predict pneumonia based on the majority class of the nearest patients.

Case-Based Learning

Case-Based Learning stores previous complete cases along with their solutions. When a new medical case is received, the system retrieves similar cases and uses their previous diagnoses to solve the new problem.

For example, if a previous patient had similar symptoms and was diagnosed with pneumonia, the system can use that case as a reference for the new patient.

Memory Usage

KNN normally stores a large portion of the training dataset because it needs the data to calculate distances during prediction. Therefore, its memory requirement can be high.

CBL also requires stored cases, but it can store **important or representative cases** instead of every training example. Therefore, its memory can be managed using case selection and removal techniques.

Similarity Measures

KNN commonly uses mathematical distance measures such as:

- Euclidean distance
- Manhattan distance
- Minkowski distance

CBL can use more flexible similarity measures based on the application. In medical diagnosis, symptoms, age, test results and medical history can be given different importance.

Prediction Strategy

KNN predicts the class using the majority vote of the nearest neighbours.

CBL follows a **retrieve, reuse, revise and retain** approach. It retrieves similar cases, reuses their solutions, modifies the solution if required, and stores the new experience.

Conclusion

KNN is simple and effective when numerical features and a suitable distance measure are available. CBL is more suitable when previous experiences and domain knowledge are important. For medical diagnosis, both can be useful, but CBL provides greater flexibility for adapting previous medical cases.

2. Compare Locally Weighted Regression and Radial Basis Function networks for function approximation tasks. Use an example dataset to explain differences in locality, computational cost, and smoothness of predictions.

Answer :

Introduction

Locally Weighted Regression (LWR) and Radial Basis Function (RBF) networks are machine learning techniques used for **function approximation**. They can be used to predict a continuous output from input data.

Consider a dataset containing **temperature and electricity consumption**. The aim is to predict electricity consumption for a new temperature value.

Locally Weighted Regression

LWR gives greater importance to training points that are close to the query point. Distant points receive smaller weights.

A common weighting function is the Gaussian kernel:

$$w_i = e^{-2\tau^2(x-x_i)^2}$$

Here, x is the query point, x_i is a training point and τ controls the size of the local neighbourhood.

Thus, LWR focuses mainly on nearby observations when making a prediction.

Radial Basis Function Network

An RBF network is a neural network that uses **radial basis functions** in its hidden layer. Each hidden neuron has a centre, and its activation depends on the distance between the input and the centre.

A commonly used RBF function is:

$$\phi(x) = e^{-2\sigma^2 \|x - c\|^2}$$

where c is the centre and σ controls the spread.

The outputs of the hidden neurons are combined to produce the final prediction.

Locality

LWR is strongly local because it calculates weights for training points around every query point.

RBF networks are also local because each RBF responds strongly near its centre. However, after training, the network uses a fixed set of centres and weights.

Computational Cost

LWR has relatively low training cost because it does not build a complex global model. However, prediction can be expensive because weights and a local regression model may need to be calculated for every new query.

RBF networks have higher training cost because centres, widths and output weights must be determined. Once trained, prediction is generally faster.

Smoothness of Predictions

LWR can produce smooth predictions, but the smoothness depends strongly on the bandwidth parameter. A small bandwidth can produce highly local and irregular predictions.

RBF networks generally provide smooth function approximation because the outputs of several radial basis functions are combined.

Conclusion

LWR is useful when the relationship changes significantly in different local regions. RBF networks are better when a **smooth and reusable approximation model** is required. LWR is more flexible locally, while RBF networks can provide faster prediction after training.

3. Compare Naïve Bayes and Logistic Regression for a text classification task such as spam detection. Analyze assumptions about feature independence, probability estimation, computational efficiency, and performance on high-dimensional data. Discuss scenarios where one method outperforms the other.

Answer :

Introduction

Naïve Bayes and Logistic Regression are popular supervised learning algorithms for **text classification**. A common application is detecting whether an email is **spam or not spam**.

Words such as *free*, *prize*, *winner*, *offer*, and *discount* can be converted into numerical features and given to the classification algorithm.

Naïve Bayes

Naïve Bayes is based on **Bayes' theorem**. It calculates the probability that an email belongs to a particular class based on the features present in the email.

The basic equation is:

$$P(C|X)=P(X)P(X|C)P(C)$$

The algorithm assumes that features are conditionally independent given the class.

For example, it assumes that the occurrence of the word "free" is independent of the occurrence of "offer" when the class is known.

Logistic Regression

Logistic Regression predicts the probability of a class using a logistic or sigmoid function.

$$P(y=1)=\frac{1}{1+e^{-z}}$$

where z is a weighted combination of the input features.

Unlike Naïve Bayes, Logistic Regression does **not require the features to be independent**.

Feature Independence

The major difference is the assumption about feature independence.

Naïve Bayes assumes that the features are independent. Although this assumption is often unrealistic for natural language, the algorithm still performs well in many text classification problems.

Logistic Regression can consider the combined effect of different features and does not make the independence assumption.

Probability Estimation

Naïve Bayes directly estimates probabilities from the training data.

Logistic Regression learns weights for the features and uses the sigmoid function to estimate the probability of a class.

Computational Efficiency

Naïve Bayes is generally very fast to train and predict. It is particularly useful when the dataset contains a very large number of words.

Logistic Regression usually requires iterative optimization, so its training can be more computationally expensive.

High-Dimensional Data

Both algorithms work well with high-dimensional text data. Naïve Bayes is especially effective when there are thousands of word features.

Logistic Regression can also perform very well when suitable regularization techniques are used.

When Naïve Bayes Performs Better

Naïve Bayes can be preferred when:

- The dataset is small.
- Fast training is required.
- There are many word features.
- A simple model is required.

When Logistic Regression Performs Better

Logistic Regression can perform better when:

- Features are correlated.
- A more flexible decision boundary is required.
- There is sufficient training data.
- Feature weights need to be interpreted.

Conclusion

Naïve Bayes is fast, simple and highly suitable for text classification. Logistic Regression is more flexible because it does not assume feature independence. The better method depends on the dataset and the relationship between the features.

4. Analyze the differences between Support Vector Machines (SVM) and KNN in classification tasks. Use an example dataset to compare decision boundaries, scalability, sensitivity to noise, and computational complexity. Discuss how kernel functions in SVM improve performance for non-linear problems.

Answer :

Introduction

Support Vector Machine (SVM) and K-Nearest Neighbour (KNN) are supervised classification algorithms. Consider a dataset containing two types of flowers based on **petal length and petal width**.

The objective is to classify a new flower into the correct category.

Support Vector Machine

SVM attempts to find an optimal **hyperplane** that separates different classes. It tries to maximize the margin between the classes.

The data points closest to the decision boundary are called **support vectors**.

A larger margin generally helps SVM achieve better generalization on unseen data.

K-Nearest Neighbour

KNN does not create a fixed model during training. When a new flower is given, it calculates the distance between the new flower and the training examples.

The K closest examples are selected, and the class with the highest number of neighbours is assigned to the new flower.

Decision Boundaries

SVM creates an explicit decision boundary during training.

KNN creates an implicit and flexible decision boundary based on the distribution of nearby data points. Therefore, KNN can produce complex boundaries without explicitly learning them.

Scalability

SVM can work effectively with high-dimensional datasets, although training can become expensive for very large datasets.

KNN becomes slower as the number of training examples increases because it must calculate distances to training points during prediction.

Sensitivity to Noise

KNN can be sensitive to noisy data because a noisy or incorrectly labelled neighbour can affect the classification.

SVM can handle some noise using a **soft-margin approach**, although it can still be affected by extreme outliers.

Computational Complexity

KNN has almost no training cost because it mainly stores the training data. However, prediction can be expensive.

SVM requires more computational effort during training to find the optimal decision boundary, but prediction is usually much faster.

Kernel Functions in SVM

One major advantage of SVM is the use of **kernel functions**.

Common kernels include:

- Linear kernel
- Polynomial kernel
- RBF kernel
- Sigmoid kernel

The **RBF kernel**, for example, allows SVM to handle non-linear relationships by effectively mapping data into a higher-dimensional feature space.

Thus, SVM can create complex decision boundaries for data that cannot be separated using a straight line.

Conclusion

KNN is simple and requires very little training, but its prediction cost increases with dataset size. SVM requires more training but provides strong classification performance and can handle non-linear data using kernel functions. Therefore, SVM is often preferred for high-dimensional and complex classification tasks.

5. Compare K-Means Clustering and Hierarchical Clustering using a dataset with unknown structure. Analyze differences in cluster formation, computational cost, scalability, and interpretability. Discuss how each method handles varying cluster shapes and the impact of choosing the number of clusters.

Answer :

Introduction

K-Means and Hierarchical Clustering are unsupervised learning algorithms used to group similar data points into clusters.

Consider a customer dataset containing age, income and spending score. The objective is to identify different customer groups without predefined class labels.

K-Means Clustering

K-Means requires the number of clusters, represented by K, to be specified before clustering.

The algorithm:

1. Selects K initial centroids.
2. Assigns each data point to the nearest centroid.
3. Calculates new centroids.
4. Reassigns data points.
5. Repeats until the clusters become stable.

For example, customers can be divided into three groups:

- Low-spending customers
- Medium-spending customers
- High-spending customers

Hierarchical Clustering

Hierarchical Clustering creates a hierarchy of clusters.

There are two main approaches:

- Agglomerative: Starts with individual points and repeatedly merges similar clusters.
- Divisive: Starts with one large cluster and repeatedly divides it.

The result can be represented using a dendrogram.

Cluster Formation

K-Means forms clusters around centroids. Each data point belongs to the cluster whose centroid is closest.

Hierarchical Clustering forms groups based on the similarity or distance between data points and clusters.

Computational Cost

K-Means is generally computationally efficient and can handle large datasets relatively well.

Hierarchical Clustering requires more computational resources because it repeatedly calculates distances and builds a hierarchy.

Scalability

K-Means is more scalable and is suitable for large datasets.

Hierarchical Clustering is usually more suitable for small or medium-sized datasets because its computational cost increases significantly with the number of data points.

Interpretability

K-Means provides a simple final grouping based on centroids.

Hierarchical Clustering is highly interpretable because the dendrogram shows how individual points and clusters are related at different levels.

Handling Different Cluster Shapes

K-Means generally performs best when clusters are relatively spherical and similar in size.

Hierarchical Clustering can identify more varied cluster structures depending on the linkage method used. However, it is not automatically suitable for every irregular shape.

Impact of Choosing Number of Clusters

The number of clusters must be selected before running standard K-Means. Choosing an inappropriate K can produce poor clusters. Techniques such as the Elbow Method and Silhouette Score can help select K.

Hierarchical Clustering does not require the number of clusters initially. The desired number can be selected later by cutting the dendrogram at an appropriate level.

Conclusion

K-Means is faster, simple and scalable for large datasets, while Hierarchical Clustering provides a detailed hierarchy and is useful when the number of clusters is unknown. The choice depends on the dataset size, cluster structure and the required level of interpretation.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	15	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer

Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	20	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	15	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand